

Access Modifiers in Java :

In order to define the **accessibility level** of the class as well as member of the class we have 4 access modifiers:

- 1) private (Within the same class)
- 2) default (Within the same package)
- 3) protected (Within the same package OR even from another package by using Inheritance)
- 4) public (No Restriction)

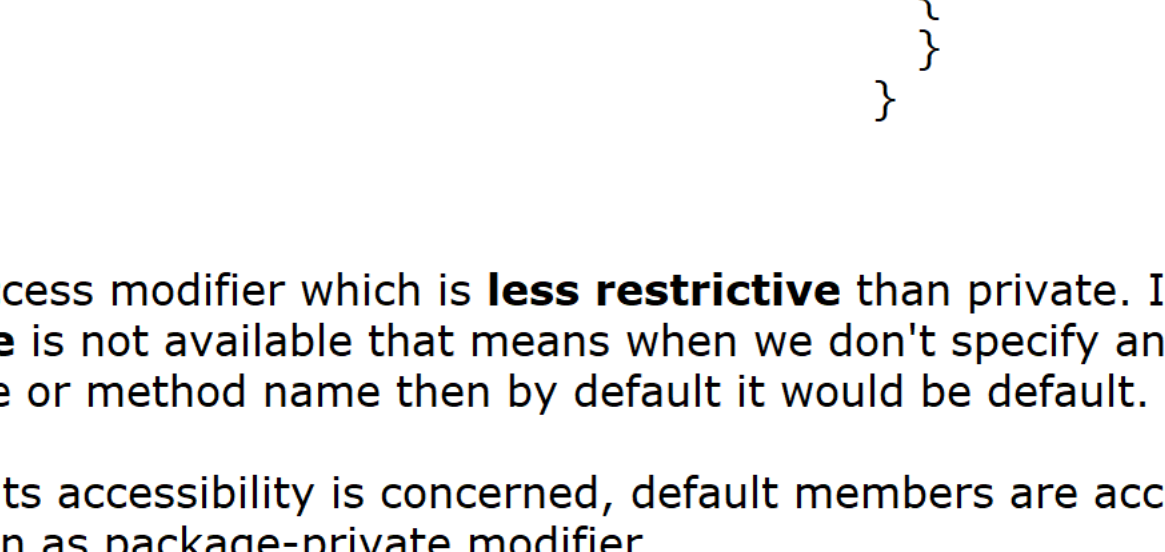
private:

It is the **most restrictive** access modifier because the member declared as private can't be accessible from outside of the class.

In Java we can't declare an **outer class** as a private or protected or static. Generally, we should declare the non-static fields with private access modifier [Data hiding].

In Java outer classes can be declared with public, abstract, final, sealed modifier only.

default:



It is an access modifier which is **less restrictive** than private. It is such kind of access modifier whose **physical existence** is not available that means when we don't specify any kind of access modifier before the class name, field name or method name then by default it would be default.

As far as its accessibility is concerned, default members are accessible within the same folder(package) only. It is also known as package-private modifier.

protected:

It is an access modifier which is less restrictive than default because the member declared as protected can be accessible from the outside of the package (folder) too but by using inheritance.

Test.java [BLC Package]

```
package com.ravi.blc;
```

```
public class Test
{
    protected int x = 2500;
}
```

Main.java [ELC package]

```
package com.ravi.elc;
```

```
import com.ravi.blc.Test;
```

```
public class Main extends Test //Inheritance
{
    public static void main(String[] args)
    {
        Main main = new Main();
        IO.println(main.x); //x is protected so accessible through Inheritance only
    }
}
```

public:

It is an access modifier which does not contain **any kind of restriction** that is the reason the member declared as public can be accessible from **everywhere** without any restriction.

According to Object Oriented rule we should declare the classes and methods as public whereas fields must be declared as private or protected according to the requirement.

Note: If a method is used for internal purpose only (like validation OR Logic calculation) then we can declare that method as private method. It is called Helper method.

Helper Method Example :

Example 1 :

```
class Test
{
    void main()
    {
        int number = Integer.parseInt(IO.readLine("Enter a number :"));
        IO.println("Is "+number+" strong number :"+StrongNumber.isStrong(number));
    }
}

class StrongNumber
{
    public static boolean isStrong(int num)
    {
        int temp = num;
        int sum = 0;

        while(num !=0)
        {
            int digit = num % 10;
            sum = sum + getFactorial(digit);
            num = num /10;
        }
        return sum==temp;
    }
}

private static int getFactorial(int num) //Helper Method
{
    int fact = 1;

    for(int i=1; i<=num;i++)
    {
        fact = fact * i;
    }
    return fact;
}
```

Example 2 :

```
public void makePayment(double amount, String creditCard, String name)
{
    if(validateCard(creditCard))
    {
        //
    }
}
```

```
public void makePayment(double amount, String debitCard)
{
    if(validateCard(debitCard))
    {
        //
    }
}
```

```
private boolean validateCard(String card) //Helper Method
{
    if(card.length()!=16)
    {
        IO.println("Invalid Card");
        return false;
    }
}
```

Diagram :

Access Modifiers	Within the same class	Within the same Package	From Another Package	From everywhere (No restriction)
private	YES	NO	NO	NO
default	YES	YES	NO	NO
protected	YES	YES	YES [Using Inheritance]	NO
public	YES	YES	YES	YES

How to print the non static fields (Object properties) using toString() method :

If we want to print our object properties (Instance Variables OR Non static fields) then we should generate(override) toString() method in our class from Object class.

Now with the help of toString() method **we need not** to write any display kind of method to print the object properties i.e. instance variable.

In order to generate the toString() method we need to follow the steps
Right click on the program -> source -> generate toString() -> generate

In order to call this toString() method, we need to print the corresponding object reference by using IO.println() statement.

```
Manager m = new Manager();
IO.println(m);//Calling toString() method of Manager class
```

```
Employee e = new Employee();
IO.println(e);//Calling toString() method of Employee class
```

```
package com.ravi.oop;
```

```
public class CarDemo
```

```
{
    void main()
    {
        Car naxon = new Car();
        naxon.setCarDetails(2026, "White", "Tata Naxon");
        IO.println(naxon); //Will call toString() method of Car class
    }
}
```

```
class Car
```

```
{
    private int carModel;
    private String carColor;
    private String carName;

    public void setCarDetails(int model, String color, String name)
    {
        carModel = model;
        carColor = color;
        carName = name;
    }

    public String toString()
    {
        return "Car [carModel=" + carModel + ", carColor=" + carColor + ", carName=" + carName + "];"
    }
}
```